

encoder_top.sv + item_mem.sv — EMG-Window Encoder and Co-Simulation Flow

This note documents the **integration block** of the 1024-HDC core: the encoder that turns one quantized EMG window into a single query hypervector. It is the piece that chains the three previously verified primitives — **XOR bind**, **permute**, and **majority bundle** — together with the **item memories**, and it is checked bit-for-bit against the Python golden reference.

1. What the encoder computes

A quantized EMG window is a grid of level indices: `n_channels` x `n_features` (here $4 \times 5 = 20$ values, each in `[0, n_levels)` with `n_levels = 16`). The encoder produces one 1024-bit query hypervector, exactly matching `hdc_ref.HDCEngine.encode_emg_window`:

```
for c in channels (4):
  for f in features (5):
    level = window[c][f]                # quantized value, 0..15
    pair  = channel[c]
        XOR value[level]
        XOR permute(feature[f], mode=2, param=f)
query = majority_bundle( all 20 pairs )
```

Three ideas combine here:

- **Bind (XOR)** ties a *role* to a *filler*: which channel, which quantized value, and which feature position.
 - **Permute** (cyclic rotation by the feature index `f`) makes feature position matter — it stops `feature[f]` from colliding with the same vector used elsewhere, encoding position information.
 - **Bundle (majority vote)** superposes all 20 role-filler pairs into one hypervector that is similar to each of its parts — the window's signature.
-

2. The RTL

item_mem.sv — hypervector ROM

A read-only memory of `N_ENTRIES` D-bit hypervectors, initialised from a `$readmemh .mem` file whose layout matches `hdc_ref.ItemMemory.export_mem_files` (one 64-bit word per line, `WORDS` lines per entry). Combinational read: drive `rd_idx`, get the full 1024-bit vector on `rd_vec`. Three instances hold the **channel** (4), **feature** (5), and **value/CiM** (16) tables. (Runtime-loadable via AXI is a later option; for now the offline-trained tables are loaded by `.mem` init, which is exactly what the co-simulation needs.)

encoder_top.sv — the datapath + FSM

Signal	Dir	Width	Meaning
<code>clk</code> , <code>rst_n</code>	in	1	clock / async reset
<code>start</code>	in	1	begin encoding the window on <code>levels_flat</code>
<code>levels_flat</code>	in	<code>N_PAIRS*LEVEL_W</code> (80)	level grid; <code>level[p]</code> at <code>[p*LEVEL_W +: LEVEL_W]</code> , pair <code>p = c*N_FEAT + f</code>
<code>busy</code>	out	1	high during encode
<code>out_valid</code>	out	1	pulses with the finished query
<code>query_vec</code>	out	<code>D</code> (1024)	bundled query hypervector

Internally the encoder instantiates three `item_mem` ROMs and one verified `bundle_unit`, plus a combinational `perm_feat()` that reproduces `permute_stage.sv` mode `2'b10` (full-D rotation). The FSM walks the 20 pairs:

```

S_IDLE --start--> S_CLR --> S_ACC (x N_PAIRS) --> S_DONE --> S_IDLE
                |           |           |
                clear bundle accumulate pair[p] latch query_vec,
                                (one per clock)   pulse out_valid

```

Each `S_ACC` cycle decodes `c = p / N_FEAT`, `f = p % N_FEAT`, reads the three ROMs, computes `pair = channel ^ value ^ permute(feature, f)`, and feeds it to `bundle_unit`. After the 20th accumulate, `S_DONE` reads the combinational majority output and emits the query. Encode latency is `N_PAIRS` + a few control cycles.

3. Verification methodology — "Python is truth"

Same pattern as the other blocks: the Python golden emits stimulus + expected output and the **actual item-memory contents**; the testbench replays them and compares bit-for-bit. There is no hand-written SV golden.

Golden vectors — `python_ref/generate_vectors.py --encoder`

`hdc_ref.export_encoder_cosim` builds the deterministic `ItemMemory`, writes the three `.mem` files, then for `count` random windows writes:

File	Lines	Contents
<code>item_mem_channel.mem</code>	<code>N_CH*WORDS</code>	channel item HVs (loaded by the DUT ROM)
<code>item_mem_feature.mem</code>	<code>N_FEAT*WORDS</code>	feature item HVs
<code>item_mem_value.mem</code>	<code>N_VAL*WORDS</code>	continuous value (CiM) HVs
<code>enc_levels.hex</code>	<code>count</code>	packed level grid (20 nibbles per case)

File	Lines	Contents
enc_expect.hex	count · WORDS	golden query hypervector
meta.txt	—	count, D, n_channels, n_features, n_levels, level_w, seed

Because the RTL ROMs and the golden read the *same* .mem files, the item memories are guaranteed identical — the test isolates the encoder datapath/FSM, not the (separately reproducible) random tables.

Testbench — tb/tb_encoder_cosim.sv and harness sim/run_encoder_cosim.do

Per case the TB drives levels_flat , pulses start , waits for out_valid , and compares query_vec against enc_expect.hex . Any mismatch prints the case and first differing bit and ends with \$fatal ; a clean sweep ends with \$finish .

```
vsim -c -do sim/run_encoder_cosim.do          # NUM_CASES env var overrides count
(default 500)
```

Co-simulation flow

generate_vectors.py --encoder	tb_encoder_cosim.sv	encoder_top.sv
-----	-----	-----
ItemMemory -> item_mem_*.mem		item_mem ROMs
(init)		
random windows	-> \$readmemh enc_levels.hex	
encode_emg_window (golden)	-> enc_expect.hex	
	drive levels, start	-> FSM: 20x
(bind+permute)		
		-> bundle_unit ->
query		
	compare query_vec	<- query_vec /
out_valid		
	PASS (\$finish) / FAIL (\$fatal)	

4. Result

```
tb_encoder_cosim: EMG-window encoder vs Python golden
VECDIR = python_ref/vectors/cosim_encoder
CASES = 500 (D=1024, 4x5 pairs)
PASS: all 500 encoder cases match the Python golden bit-for-bit.
```

500 / 500 cases pass on ModelSim SE-64 10.6e. item_mem.sv , encoder_top.sv , tb_encoder_cosim.sv , and the encoder data files are added to the ModelSim project (1024HDC.mpf).

This is the first **multi-block integration** verified end-to-end: it composes the XOR-bind+permute datapath, the majority bundler, and the item memories into the encoder that produces the query hypervector. The

remaining step to a full inference core is to wire `encoder_top` to the already-verified `popcount_am` (plus the pruning mask) in a single `hdc_core_top`, then expose it over AXI.